

جامعة الحسن الثاني بالدار البيضاء
كلية العلوم بن مسيك



FACULTE DES SCIENCES BEN M'SICK
UNIVERSITÉ HASSAN II DE CASABLANCA

Travaux Diriger Programmation II Langage C



SMI S4

Pr AIT DAOUD

Pr OUNACER

EXERCICE 1

Ecrire un programme qui permet à l'utilisateur de remplir dynamiquement un tableau de n éléments et puis d'utiliser les fonctions suivantes :

1. Une fonction qui permet de trier les éléments du tableau par ordre croissant.
2. Une fonction qui permet de trier les éléments du tableau par ordre décroissant.

Solution

```
#include <stdio.h>
#include <stdlib.h>

void triect(int *tab,int taille) //fonction trie croissant
{
    int i,j,tmp;

    printf("les éléments du tableau sont:\n");
    for(i=0;i<taille;i++){
        printf("%d\t",*(tab+i));
    }
    for(i=0;i<taille;i++){
        for(j=i+1;j<taille;j++){
            if(*(tab+i)>*(tab+j)){
                tmp=*(tab+i);
                *(tab+i)=*(tab+j);
                *(tab+j)=tmp;
            }
        }
    }
    printf("\nles éléments du tableau par ordre croissant sont:\n");
    for(i=0;i<taille;i++){
        printf("%d\t",*(tab+i));
    }
}

void triedt(int *tab,int taille) //fonction tri décroissant
{
    int i,j,tmp;
    for(i=0;i<taille;i++){
        for(j=i+1;j<taille;j++){
            if(*(tab+i)<*(tab+j)){
                tmp=*(tab+i);
                *(tab+i)=*(tab+j);
                *(tab+j)=tmp;
            }
        }
    }
    printf("\nles éléments du tableau par ordre décroissant sont:\n");
    for(i=0;i<taille;i++){
        printf("%d\t",*(tab+i));
    }
}

main()
{
    int *t,n,i;
```

```
do{
    printf("veuillez saisir la taille du tableau \n");
    scanf("%d",&n);
}while(n<=0);

t=(int*)malloc(n*sizeof(int));

for(i=0;i<n;i++)
{
    printf("t[%d]= ",i+1);
    scanf("%d",t+i);
}

triect(t,n);
triedt(t,n);
free(t);

}
```

EXERCICE 2

Ecrire un programme, qui saisit deux tableaux d'entiers positifs A et B et qui copie alternativement leurs éléments dans un troisième tableau Tab. En utilisant les fonctions remplir(), copier() et afficher().

Travail à faire

Tableau A :

1	2	3	4	5	6	7
---	---	---	---	---	---	---

Tableau B :

10	20	30
----	----	----

Tableau Tab :

1	10	2	20	3	30	4	5	6	7
---	----	---	----	---	----	---	---	---	---

Solution :

```
#include <stdio.h>
#include <stdlib.h>

void remplir(int *t, int n) //fonction remplissage tableau
{
    int i;
    for(i=0;i<n;i++){
        printf("entrer t[%d]= ",i+1);
        scanf("%d",t+i);
    }
}

void copier(int *TAB,int *A,int*B, int n,int m) //fonction copier
{
    int i,j=0,k=0;
    for(i=0;i<n+m;i+=2)
    {
        if(j<n && k<m)
        {
            *(TAB+i)=*(A+j);
            *(TAB+i+1)=*(B+k);
            j++;
            k++;
        }
        else {
            if(k<m){
                *(TAB+i)=*(B+k);
                *(TAB+i+1)=*(B+k+1);
                k=k+2;
            }
            else {
                *(TAB+i)=*(A+j);
                *(TAB+i+1)=*(A+j+1);
                j=j+2;}
        }
    }
}
```

```
    }  
  }  
}  
  
void afficher(int *t, int a) //fonction affichage tableau  
{  
  int i;  
  for(i=0;i<a;i++){  
    printf("%d\t",*(t+i));  
  }  
  printf("\n\n");  
}  
main()  
{  
  int *A,*B,*TAB;  
  int tailleA,tailleB;  
  printf("donner la taille des deux tableaux A et B: \n");  
  scanf("%d %d",&tailleA,&tailleB);  
  A=(int*)malloc(tailleA*sizeof(int));  
  B=(int*)malloc(tailleB*sizeof(int));  
  TAB=(int*)malloc((tailleA+tailleB)*sizeof(int));  
  
  printf("Remplir le tableau A\n");  
  remplir(A,tailleA);  
  
  printf("Remplir le tableau B\n");  
  remplir(B,tailleB);  
  
  copier(TAB,A,B,tailleA,tailleB);  
  
  printf("Tableau A\n");  
  afficher(A,tailleA);  
  
  printf("Tableau B\n");  
  afficher(B,tailleB);  
  
  printf("Tableau TAB\n");  
  afficher(TAB,(tailleA+tailleB));  
  
  free(A);  
  free(B);  
  free(TAB);  
}
```

EXERCICE 3

En utilisant la récursivité, écrire un programme qui permet de déterminer le nombre de combinaisons possibles de p éléments parmi n sachant que C_n^p peut-être calculer par :

$$C_n^p = C_{n-1}^{p-1} + C_{n-1}^p$$

$$C_n^n = 1$$

$$C_n^1 = n$$

Solution :

```
#include <stdio.h>
#include <stdlib.h>
```

```
int combinaison(int p,int n)
{
    if (p==n)
        return 1;
    else if (p==1)
        return n;
    else return combinaison(p-1,n-1) + combinaison(p,n-1);
}
```

```
main()
{
    int n,p;
    do{
        printf("saisir les valeurs p et n:\n ");
        scanf("%d %d",&p,&n);
    }while(n<0 || p<0 || p>n);
    printf(" le nombre de combinaisons de %d dans %d est: %d",p,n,combinaison(p,n));
}
```

EXERCICE 4

On désire calculer la somme suivante :

$$1 + x1/1! + x2/2! + x3/3! + x4/4! + x5/5! + \dots$$

Pour cela écrire une fonction **expo** qui calcule cette somme jusqu'à un terme (terme c'est $x_i/i!$) inférieur strictement à un nombre ep ; x et ep sont des paramètres de la fonction **expo**. Bien sur cette fonction fait appel à deux fonction puissance et factorielle, qu'il faut définir à cet effet, pour calculer le terme. Tester ces fonctions.

Solution :

- 1ère méthode

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int puissance(int x,int n) //fonction puissance
```

```
{
```

```
    if(n==0)
```

```
        return 1;
```

```
    if(n==1)
```

```
        return x;
```

```
    int i, p=1;
```

```
    for(i=1;i<=n;i++)
```

```
        p*=x;
```

```
    return p;
```

```
}
```

```
int factoriel(int n) //fonction factorielle
```

```
{
```

```
    if(n==0)
```

```
        return 1;
```

```
    int f=1,i;
```

```
    for(i=n;i>=1;i--)
```

```
        f=f*i;
```

```
    return f;
```

```
}
```

```
float expo(int x,int ep){
```

```
    int i;
```

```
    float s=1;
```

```
    for(i=1;i<=ep;i++)
```

```
        s=s+(float)puissance(x,i)/(float)factoriel(i);
```

```
    return s;
```

```
}
```

```
main()
```

```
{int a,n;
```

```
    printf("%.2f \n",expo(3,2));
```

```
}
```

- 2-ème méthode (en utilisant la récursivité)

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int puissance(int x,int n)
```

```
{
    if(n==0)
        return 1;
    if(n==1)
        return x;
    else
        return x*puissance(x,n-1);
}
int factoriel(int a){
    if(a==0)
        return 1;
    else return a*factoriel(a-1);
}
float expo(int x,int ep){
    int i;
    float s=1;
    for(i=1;i<=ep;i++)
        s=s+(float)puissance(x,i)/(float)factoriel(i);
    return s;
}
main()
{int a,n;
    printf("%.2f \n",expo(3,2));
}
```


EXERCICE 5

Soit l'expression suivante ($n!$ signifie le factoriel de n) :

$$E(n) = \frac{1!}{n} + \frac{2!}{n} + \frac{3!}{n} + \dots + \frac{(n-1)!}{n} + \frac{n!}{n}$$

On désire de trouver les entiers n tel que $E(n)$ est compris entre les deux nombres $N1$ et $N2$ et d'afficher la valeur de $E(n)$ à chaque fois qu'on trouve un entier n . pour cela on demande :

1. Ecrire une fonction qui calcule le factoriel d'un entier.
2. Ecrire une fonction qui calcule la somme $1! + 2! + \dots + (n-1)! + n!$
3. Ecrire la fonction qui répond à ce qui est demandé.

Solution :

```
#include <stdio.h>
#include <stdlib.h>

int fact(int n) //fonction factorielle en utilisant la récursivité
{
    if(n==0 || n==1)
        return 1;
    else return n*fact(n-1);
}

int somme(int n) //fonction somme
{
    int i, somme=0;
    for(i=1; i<=n; i++)
        somme+=fact(i);
    return somme;
}

void En(int N1, int N2) //fonction E(n)
{float En;
int n=1;
printf("les entiers n qui vérifient la condition %d < E(n) < %d sont :\n", N1, N2);
do{
    En=(float)somme(n)/n;
    if(En>N1 && En<N2)
        {printf("E(%d)=%f\n", n, En);}
    n++;}
while(En<N2);
}

main()
{
    int N1, N2;
    do{
        printf("donner les deux nombres positif N1 et N2:");
        scanf("%d %d", &N1, &N2);
    }while(N1<0 || N2<0);
    En(N1, N2); //appel à la fonction E(n)
}
```

EXERCICE 6

On désire manipuler une liste d'entiers positifs. Ecrire les fonctions suivantes :

4. Ecrire une fonction **saisie** qui crée une liste.
5. Ecrire une fonction **recherche** qui recherche une valeur VAL entrée au clavier. Cette fonction retourne la position de VAL si elle se trouve dans la liste, sinon elle retourne la valeur -1.
6. Ecrire une fonction **frequence** qui calcule la fréquence d'une valeur VAL entrée au clavier. Cette fonction retourne le nombre de répétition de VAL dans la liste (si elle existe évidemment), sinon elle retourne la valeur 0.
7. Ecrire une fonction **remplace_18** qui remplace, dans la liste tous les nombres pairs par 18.
8. Ecrire une fonction **affiche_8** qui affiche, à partir du tableau tous les nombres qui se termine par 8.
9. Ecrire une fonction **chiffre_3** qui affiche, à partir du tableau tous les nombres qui ont trois chiffres.
10. Ecrire une fonction **inverse** qui inverse le tableau sur place.
11. Ecrire une fonction **nombre_PALI** qui supprime tous les nombres palindromes du tableau. Exemple les nombres 121 11 313 sont palindromes.
12. Ecrire une fonction **maximum** qui détermine le maximum de la liste.
13. Ecrire une fonction **minimum_der** qui détermine la dernière position du minimum de la liste.
14. Ecrire une fonction **nombre_3** qui calcule le nombre des nombres multiples de 3 puis les affiche.
15. Ecrire une fonction **moy_pair_impair** qui affiche la moyenne des nombres pairs et la moyenne impairs.
16. Ecrire une fonction **chiffre_4** qui affiche, à partir du tableau tous les nombres qui ont quatre chiffres.
17. Ecrire une fonction **tri** qui permet de trier le tableau selon l'ordre croissant.
18. Ecrire une fonction **decalage_gauche** qui décale tous les éléments d'un rang vers la gauche, le premier viendra en dernière position.
19. Exemple : 1 4 6 11 8 9 0 devient à gauche 0 1 4 6 11 8 9
20. Ecrire une fonction **decalage_droite** qui décale tous les éléments d'un rang vers la droite, le dernier viendra en première position.
21. Exemple : 1 4 6 11 8 9 0 devient à droite 0 1 4 6 11 8 9
22. Ecrire une fonction **Sup_moy** qui calcule le nombre des entiers supérieurs à la moyenne puis les affiche.
23. Ecrire une fonction **sous_suite** qui calcule la somme des sous suites des pairs et la somme des sous suites des impairs.
24. Ecrire un programme qui fait appel à toutes les fonctions précédentes. Utilisez selon que

Solution :

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// question 1
```

```
void saisie(int *liste, int n)
```

```
{ int i;
```

```
    for (i = 0; i < n; i++)
```

```
{
```

```
        printf("entrer un nombre : ");
        scanf("%d", &liste[i]);
    }
}
```

```
void affiche(int *liste, int n)
```

```
{
    int i;
    printf("[");
    for (i = 0; i < n; i++)
    {
        if (i == (n-1))
            printf("%d\\n", liste[i]);
        else
            printf("%d, ", liste[i]);
    }
}
```

// question 2

```
int recherche(int *liste, int n)
{
    int valeur, i, j;
    printf("entrer la valeur a rechercher : ");
    scanf("%d", &valeur);
    for (i = 0; i < n; i++)
        if (liste[i] == valeur)
            return i;
    return -1;
}
```

// question 3

```
int frequence(int *liste, int n)
{
    int valeur, occur = 0, i;
    printf("entrer une valeur : ");
    scanf("%d", &valeur);
```

```
    for (i = 0; i < n; i++)
        if (liste[i] == valeur)
            occur++;
    return occur;
}
```

// question 4

```
void remplace_18(int *liste, int n)
{
    int i;
    for (i = 0; i < n; i++)
        if (liste[i] % 2 == 0)
            liste[i] = 18;
}

int terminePar8(int nbr)
{
    while (nbr >= 10)
        nbr = nbr % 10;
    if (nbr == 8)
        return 1;
    return 0;
}
```

// question 5

```
void affiche_8(int *liste, int n)
{
    int i;
    for (i = 0; i < n; i++)
        if (terminePar8(liste[i]) == 1)
            printf("%d\n", liste[i]);
}
```

// question 6

```
int compterChiffres(int nbr)
{

```

```
int count = 0;

while (nbr != 0) {
    nbr /= 10;    // nbr = nbr/10
    ++count;    //count = count+1
}

return count;
}

void chiffre_3(int *liste, int n)
{
    int i;
    for (i = 0; i < n; i++)
        if (compterChiffres(liste[i]) == 3)
            printf("%d\n", liste[i]);
}
```

// question 7

```
void copieListe(int *source, int *dist, int n)
{
    int i;
    for (i = 0; i < n; i++)
        dist[i] = source[i];
}

void inverse(int *liste, int n)
{
    int i, j, *tmp = malloc(n * (sizeof(int)));
    copieListe(liste, tmp, n);
    for (j = n - 1, i = 0; j >= 0; j--, i++)
        liste[i] = tmp[j];
}
```

// question 8

```
int palindrome(int n)
{
    int r, sum=0, temp;

    temp = n;
```

```
while(n > 0)
{
    r = n%10;
    sum = (sum*10) + r;
    n= n/10;
}
if(temp == sum)
    return 1;
return 0;
}

void nombre_PALL(int *liste, int n)
{
    int i;
    for (i = 0; i < n; i++)
        if (palindrome(liste[i]) == 1)
            liste[i] = 0;
}
```

// question 9

```
int maximum(int *liste, int n)
{
    int max = liste[0], i;
    for (i = 1; i < n; ++i)
    {
        if (max < liste[i])
            max = liste[i];
    }
    return max;
}
```

// question 10

```
int minimum_der(int *liste, int n)
{
    int min = liste[0], pos = -1, i;
    for (i = 1; i < n; ++i)
```

```
{  
    if (min >= liste[i]) {  
        min = liste[i];  
        pos = i;  
    }  
}  
return pos;  
}
```

// question 11

```
void nombre_3(int *liste, int n)  
{  
    int i, occurs = 0, *tmp = malloc(n * (sizeof(int)));  
    for (i = 0; i < n; i++)  
        if (liste[i]%3 == 0)  
            tmp[occurs++] = liste[i];  
    printf("le nombre des nombres multiples de 3 dans la liste est : %d \n", occurs);  
    for (i = 0; i < occurs; i++)  
    {  
        printf("%d\n", tmp[i]);  
    }  
}
```

// question 12

```
void moy_pair_impair(int *liste, int n)  
{  
    int i, nbr_pairs = 0, nbr_impairs = 0;  
    for (i = 0; i < n; i++) {  
        if (liste[i]%2 == 0)  
            nbr_pairs++;  
        else  
            nbr_impairs++;  
    }  
    printf("le moyen de nbr pairs est %d/%d\n", nbr_pairs, n);  
    printf("le moyen de nbr impairs est %d/%d\n", nbr_impairs, n);  
}
```

// question 13

```
void chiffre_4(int *liste, int n)
{
    int i;
    for (i = 0; i < n; i++)
        if (compterChiffres(liste[i]) == 4)
            printf("%d\n", liste[i]);
}
```

// question 14

```
void tri(int *list, int n)
{
    int i, j, a;
    for (i = 0; i < n; ++i)
    {
        for (j = i + 1; j < n; ++j)
        {
            if (list[i] > list[j])
            {
                a = list[i];
                list[i] = list[j];
                list[j] = a;
            }
        }
    }
}
```

//question 15

```
void decalge_gauche(int *list, int n)
{
    int i, j, temp=list[0];
    for(i = 0; i < n; i++)
        list[i]=list[i+1];
    list[n-1]=temp;
}
```



```
}
```

//question 16

```
void decalge_droit(int *list, int n)
{
    int i,j,temp=list[n-1];
    for(i = n-1; i > 0; i--)
        list[i]=list[i-1];
    list[0]=temp;
}
```

//question 17

```
void sup_moy(int *list, int n)
{
    int i, sum, nb=0;
    float moy;
    for (i= 0 ; i <n; i++)
        sum +=list[i];
    moy = sum/n;
    for(i = 0; i < n; i++)
        if(list[i] > moy)
            nb++;
    for(i = 0; i < n ;i++)
        if(list[i] > moy)
            printf("\n la note %d", list[i]);
            printf("\n le nombre %d", nb);
}
```

// question 19

```
void appel(){
    int c,n;
    do{
        printf("Merci de choisir le numéro de la fonction que vous voulez appelez sinon entrer 0 pour
sortire");
        scanf("%d", n);
    }while (c!=0);
}
```

```
int main()
{   int n;
    do{
        printf("entrer la taille de liste : ");
        scanf("%d", &n);
    }while(n < 0);

    // dynamically allocate memory for integer liste of size n
    int *liste = malloc(n * (sizeof(int))), r;
    saisie(liste, n);
    /* r=recherche(liste, n);
    if (r!=-1)
        printf("\n la valeur rechercher est positionner %d: ",r);
    else
        printf("la valeur rechercher n'est pas trouvern \n ");
    affiche(liste, n);
    inverse(liste, n);*/
    sup_moy(liste, n);
    affiche(liste, n);
    return 0;
}
```